

2025

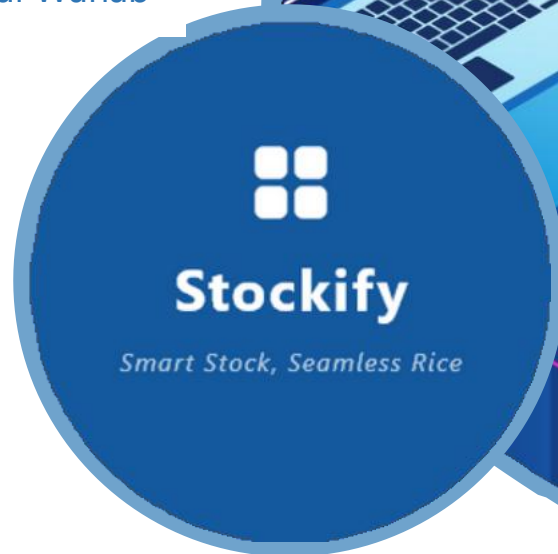
MORALVERSE

SCD

**Moral Tales Management
System**

Software Construction & Dev

Submitted to: Sir Abdul Wahab



Prepared By:

Fizza Irfan (70144509)

Amina Arif (70141893)

Areebah Mushtaq (70143646)

Moral Storytelling Characters Management System

Abstract :

This project focuses on the design and documentation of a web-based Moral Storytelling Characters Management System. The system is developed to manage characters and moral stories in an organized manner and present them to users through a storytelling interface. The backend is designed using [Flask REST APIs](#) with a [MySQL](#) database, while the frontend is developed using React. The system follows software construction principles such as modularity, abstraction, and proper documentation. [CRUD](#) (Create, Read, Update, Delete) operations are implemented on core system modules to demonstrate real-world backend system development.

Introduction :

Storytelling has always been an effective way to convey moral values and ethical lessons. With the advancement of web technologies, digital platforms can be used to manage and present stories in an interactive manner. This project aims to design a structured web application that manages moral stories and their associated characters, allowing users to explore and listen to these stories through a modern web interface.

Project Overview :

The Moral Storytelling Characters Management System is a web-based application that provides two main panels: an Admin panel and a User panel. The [Admin](#) is responsible for managing characters, stories, and user accounts, while

Users can view characters and listen to moral stories. The system is built using a REST-based architecture where React acts as the client, Flask serves as the backend API layer, and MySQL is used for data storage.

Problem Statement :

Traditional moral stories are often available in unstructured formats such as books or static content platforms, making it difficult to manage, update, or categorize stories efficiently. There is a need for a centralized system where moral stories and characters can be systematically managed and presented to users in an engaging and interactive way.

Affected Group :

- Students and children who want to learn moral values through stories
- Content creators and administrators who manage moral storytelling content
- Educational institutions seeking digital learning tools

Assumptions :

- Users have access to a modern web browser
- Internet connectivity is available
- Admin users have basic technical knowledge to manage content

Dependencies / External Systems :

- ✧ Web browser for accessing the frontend
- ✧ XAMPP for MySQL database management
- ✧ Flask framework for backend APIs
- ✧ React library for frontend development

Goals :

- ✧ To design a REST-based backend system using Flask
- ✧ To implement CRUD operations on core modules
- ✧ To store and manage moral stories and characters in a database
- ✧ To provide a simple and interactive user interface

Stakeholders :

- ❖ Admin (System Manager)
- ❖ End Users (Story Listeners)
- ❖ Course Instructor
- ❖ Development Team

Domain Analysis :

The domain of this project is educational storytelling systems. The system revolves around managing characters, stories, and users. Characters act as narrators, stories contain moral lessons, and users interact with the system to explore these stories.

Data Dictionary :

1. Characters

character_id (int, PK)
name (varchar)
role (varchar)
description (text)

2. Stories

story_id (int, PK)
title (varchar)
moral_lesson (varchar)
story_text (text)

character_id (int, FK)

3. Users

user_id (int, PK)

username (varchar)

password (varchar)

role (varchar)

Functional Requirements :

- ✧ Admin can create, update, delete, and view characters
- ✧ Admin can create, update, delete, and view stories
- ✧ Admin can manage user accounts
- ✧ Users can view characters
- ✧ Users can listen to and read moral stories

Non-Functional Requirements :

- ✧ The system should be easy to use
- ✧ The system should provide secure authentication
- ✧ The system should respond within acceptable time limits
- ✧ The system should be maintainable and modular

Tools and Technologies :

- ❖ Frontend: React (JavaScript)
- ❖ Backend: Flask (Python)
- ❖ Database: MySQL (XAMPP)
- ❖ API Communication: JSON

API Endpoints :

1. Characters

POST /api/characters

GET /api/characters

GET /api/characters/{id}
PUT /api/characters/{id}
DELETE /api/characters/{id}

2. Stories

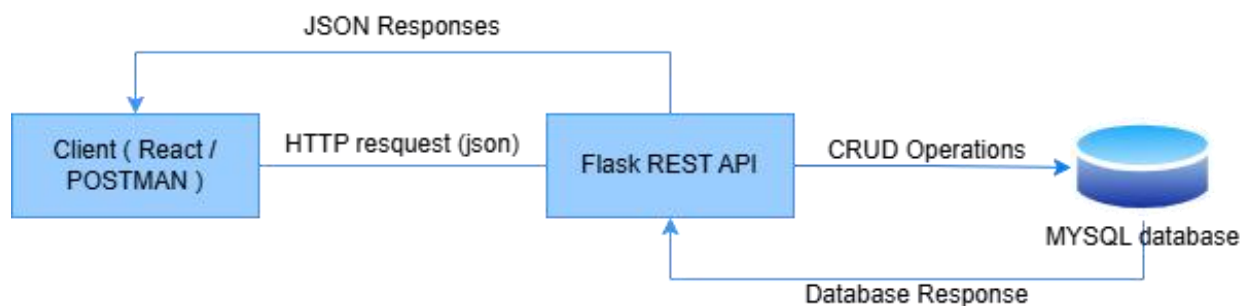
POST /api/stories
GET /api/stories
GET /api/stories/{id}
PUT /api/stories/{id}
DELETE /api/stories/{id}

3. Users

POST /api/signup
POST /api/login
GET /api/users

High Level System Architecture :

The system follows a three-tier architecture consisting of a Client layer, Backend API layer, and Database layer.



Designed by : Software engineers
Tool used : Draw.io

Architecture Description:

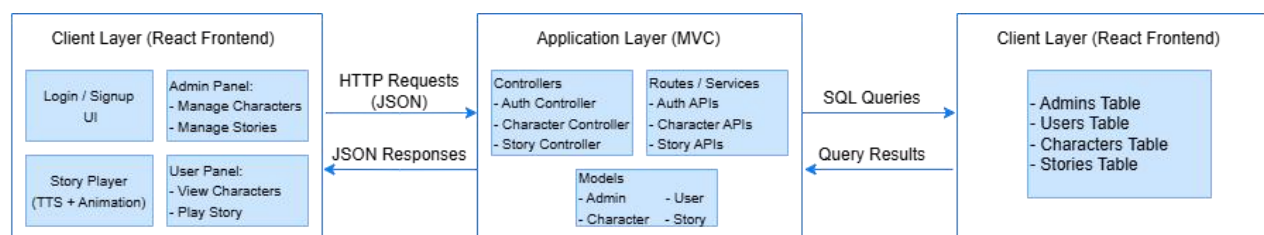
Client Layer (React / Postman) : This layer provides the graphical user interface. Users and admins interact with the system through web pages developed in React. All

requests are sent to the backend in JSON format.

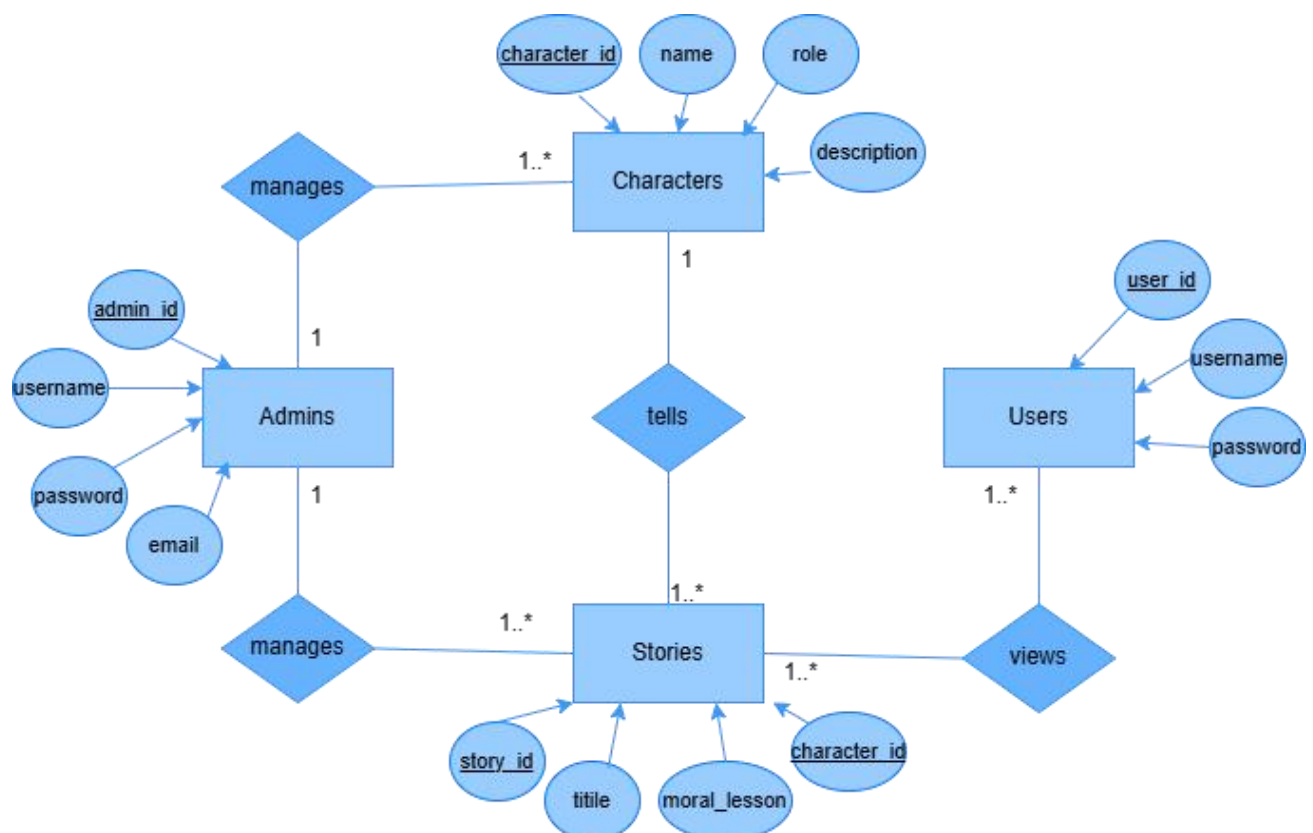
Flask REST API Layer : This layer handles all business logic. It receives requests from the client, validates data, applies application rules, and performs CRUD operations by communicating with the database.

Database Layer (MySQL) : This layer stores all persistent data such as users, characters, and stories. Flask executes SQL queries on this database.

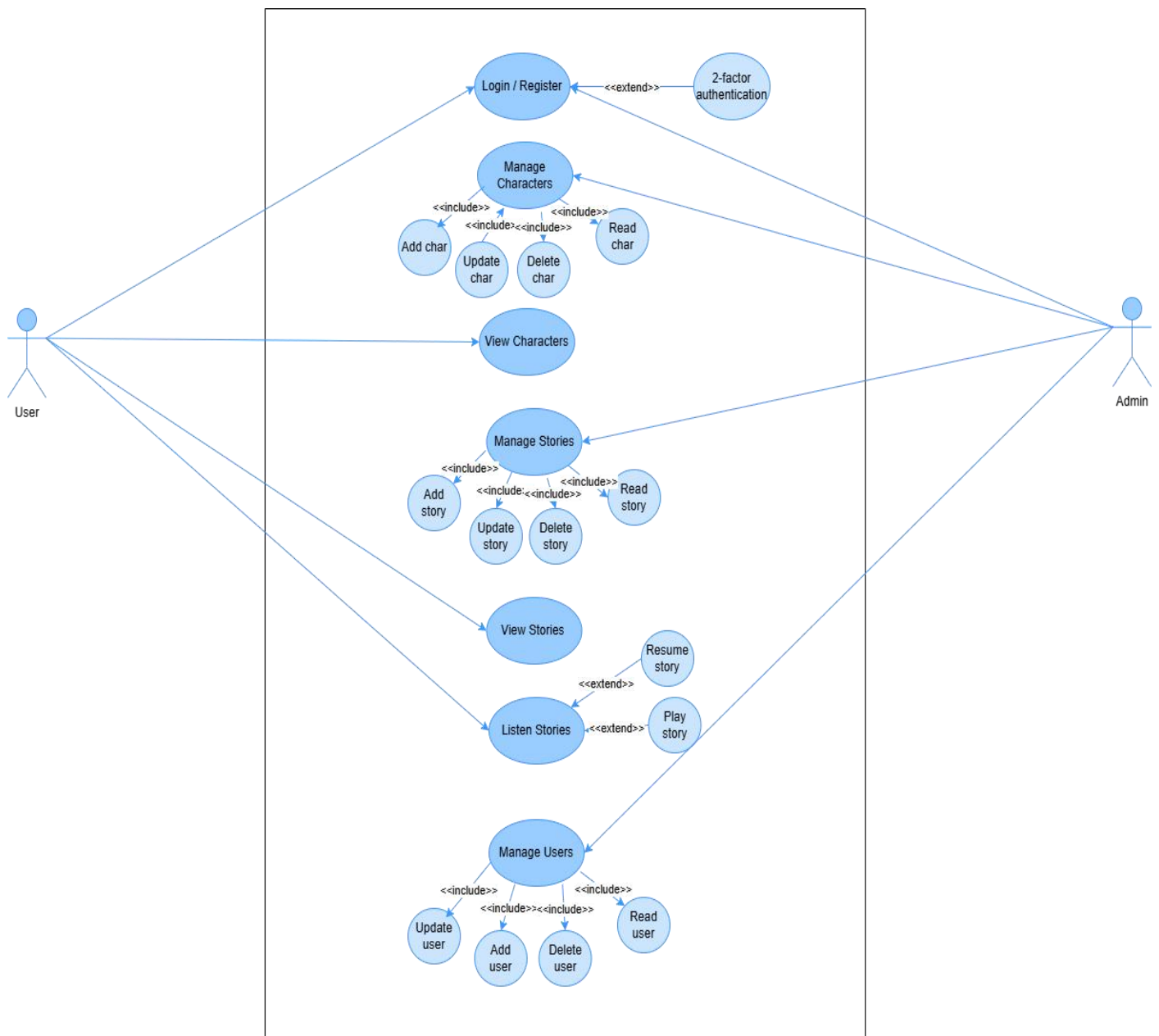
✓ Application Architecture:



Entity Relationship Diagram (ERD) :



Use Case Model :



API Section :

- ✓ Endpoint name
- ✓ HTTP method
- ✓ Purpose
- ✓ Sample request / response

Example: Character APIs

1. Create Character :

- **Endpoint:** /api/characters
- **Method:** POST
- **Description:** Creates a new character in the system.
- **Sample Request:**

```
{  
  "name": "Ayaan",  
  "role": "Narrator",  
  "description": "Wise moral storyteller"}  
Sample Response:
```

```
{  
  "message": "Character created successfully"}  
- Status Code: 201 Created
```

2. Get Characters :

- **Endpoint:** /api/characters
- **Method:** GET
- **Description:** Retrieves a list of all characters.
- **Status Code:** 200 OK

3.Story APIs : (Create Story)

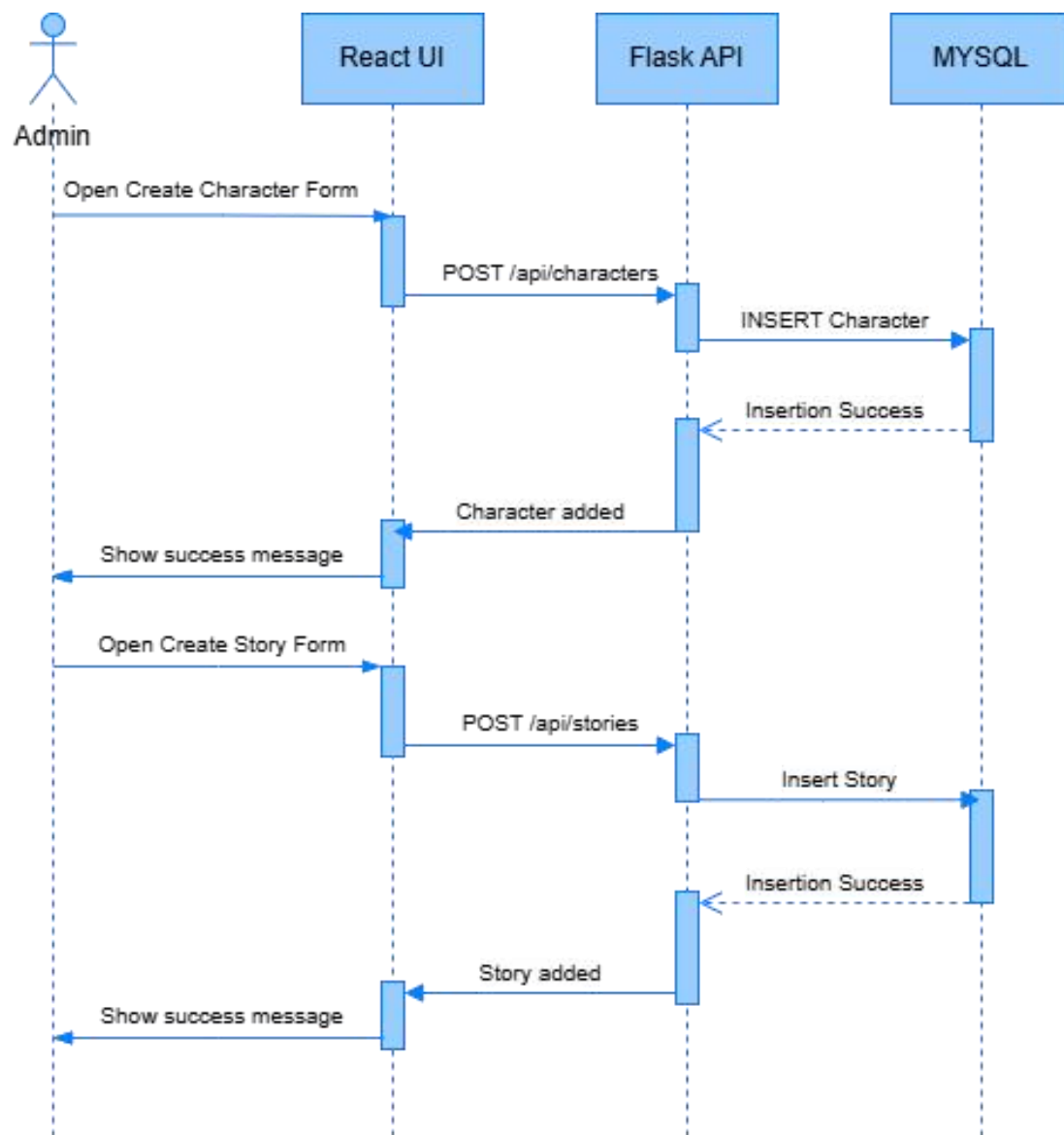
- **Endpoint:** /api/stories
- **Method:** POST
- **Description:** Adds a new moral story linked with a character.

4.Authentication API : (Login)

- **Endpoint:** /api/login
- **Method:** POST
- **Description:** Authenticates admin or user and grants access based on role.

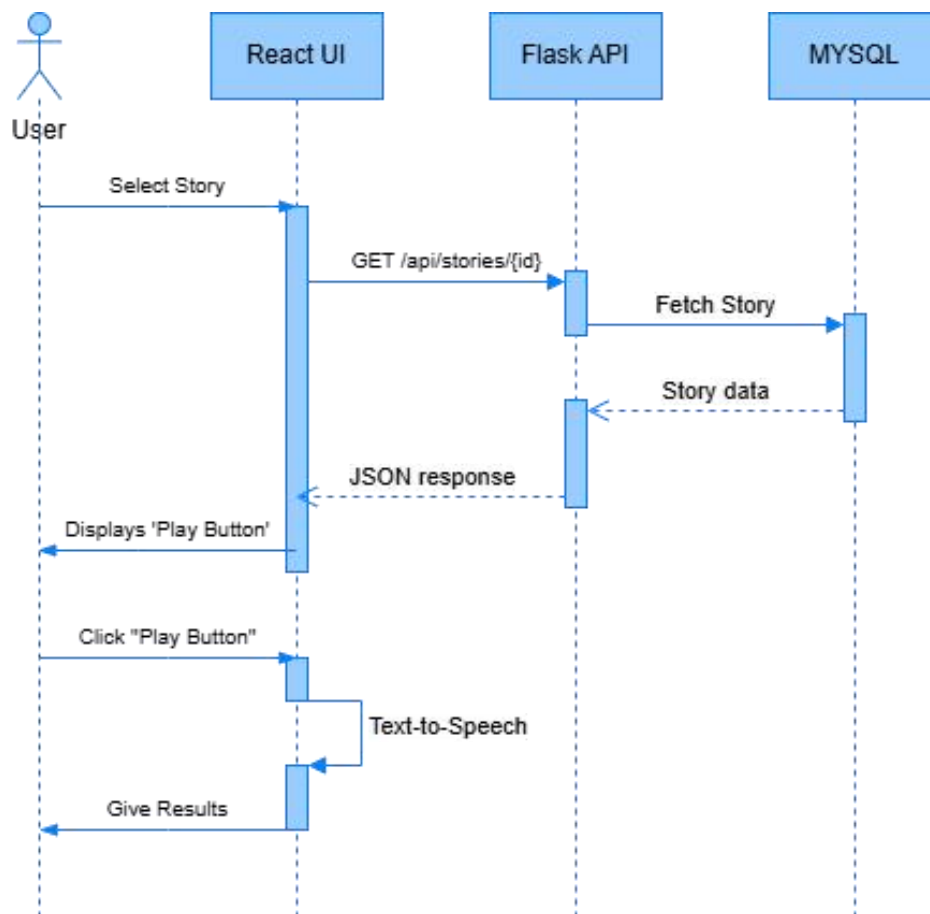
Sequence Diagram:

- ✓ Scenario: "Admin Story & Character CRUD (Create only) Sequence Diagram"

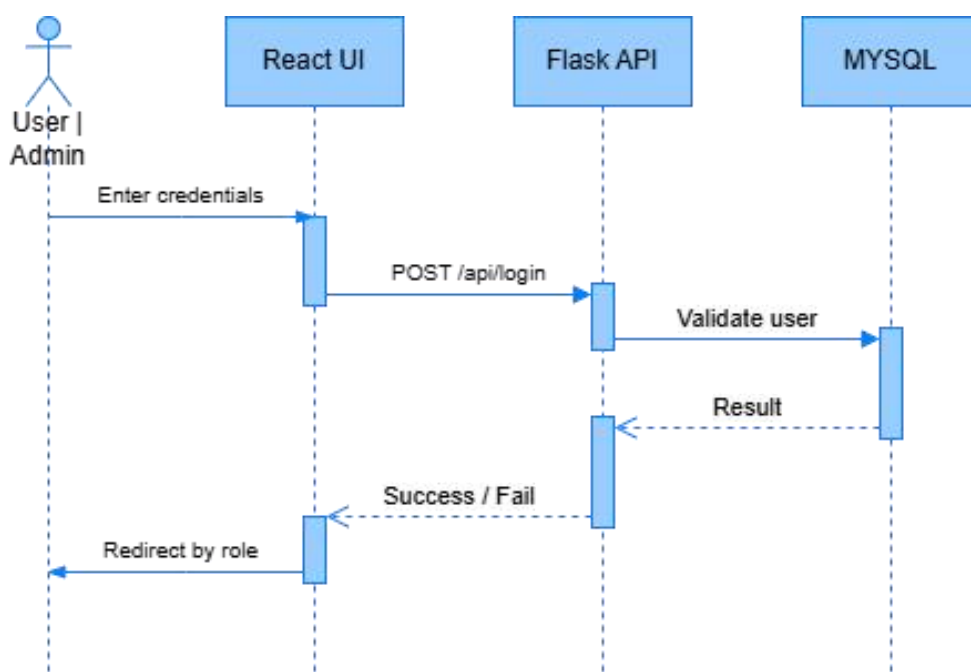


Admin Story & Character CRUD (Create) –
Sequence Diagram

✓ Scenario: "Story Play Sequence Diagram"



✓ Scenario: "Login Sequence Diagram"



Security Architecture :

The system implements basic security mechanisms to protect user data and prevent unauthorized access. Authentication is handled through a [login system](#) where user credentials are [validated](#) on the backend. [Role-based access control](#) is implemented to ensure that only admin users can perform CRUD operations, while normal users are limited to viewing and listening to stories. Passwords are stored in [encrypted or hashed form](#), and API endpoints are protected to ensure secure communication between the frontend and backend.

Tools & Tech:

Tools and Technologies		
Category	Tool / Tech	Purpose
Frontend	React.js	Used to build the user interface and handle client-side logic
Backend	Flask (Python)	Used to develop RESTful APIs and application logic
Database	MySQL (XAMPP)	Used for storing persistent data such as users, stories, and characters
API Testing	Postman	Used for testing and validating REST API requests and responses
Animation	Lottie / CSS Animations	Used for displaying 2D animated storytelling characters
Text-to-Speech	Browser SpeechSynthesis API	Used to convert story text into voice narration
Diagram Tool	draw.io	Used for creating UML and system architecture diagrams
Version Control	Git	Used for managing source code versions
Development Environment	VS Code	Used for writing and managing project source code

Conclusion :

The Moral Storytelling Characters Management System provides a structured approach to managing and presenting moral stories through a modern web application. By following software construction principles and REST architecture, the system ensures modularity, scalability, and maintainability.
